

File: README.md

Enterprise-Grade RAG Platform

A full-stack retrieval-augmented generation (RAG) platform built with a FastAPI backend, Next.js frontend, MongoDB storage, and Mistral-powered AI search.

Overview

This repository delivers a deployable enterprise knowledge platform with:

- Authenticated document upload and management
- Semantic search and chat-based question answering
- Contextual answer citations with transparency
- Prompt injection detection and security logging
- Analytics, feedback, and audit dashboards
- Local Docker Compose and cloud deployment support

Current Configuration

The backend is configured to use a single LLM provider:

- `Mistral` as the only active model provider
- `backend/runtime.txt` pins Python to `3.13`
- No fallback provider chain is required in production

Repository Structure

```
```text
backend/
 app/
 api/v1/routes/
 core/
 db/
 models/
 schemas/
 services/
 workers/
 tests/
frontend/
 app/
 components/
 lib/
 styles/
docs/
infra/
```
```

Quick Start

1. Copy `.env.example` to `.env`.
2. Set `MISTRAL\_API\_KEY` and `MONGO\_URI`.
3. Install backend dependencies:

```
```bash
cd backend
python -m pip install -r requirements.txt
```
```

4. Install frontend dependencies:

```
```bash
cd ../frontend
npm install
```
```

5. Run services locally:

```
```bash
docker compose up --build
```
```

6. Access the application:

- Frontend: `http://localhost:3000`
- Backend: `http://localhost:8000`

Environment Variables

Key variables are documented in `.env.example` and include:

- `MISTRAL\_API\_KEY`
- `MISTRAL\_MODEL`
- `MONGO\_URI`
- `SECRET\_KEY`
- `NEXT\_PUBLIC\_API\_BASE\_URL`

Documentation

The project now includes expanded documentation to support development, deployment, and submission.

- `docs/Project\_Documentation.md`
- `docs/System\_Architecture.md`
- `docs/User\_Manual.md`
- `docs/Deployment\_Guide.md`
- `docs/Prompt\_Document.md`
- `docs/Presentation\_Content.md`
- `docs/Final\_Submission\_Checklist.md`
- `screenshots/SCREENSHOT\_CHECKLIST.md`

Deployment

Local deploy

```
```bash
docker compose up --build
```
```

Cloud deploy

- Render backend configured in `infra/render.yaml`
- Frontend is Vercel-ready with `frontend/next.config.ts`
- Use `backend/runtime.txt` to keep Python 3.13

Notes

- The current vector layer is an in-memory semantic search store in `backend/app/db/vector.py`.
- Production-grade vector storage should replace this with a dedicated vector database.

- Prompt injection detection is enforced by ``backend/app/services/rag.py``.

API Endpoints

- OpenAPI schema: ``http://localhost:8000/api/v1/openapi.json``
- Swagger UI: ``http://localhost:8000/docs``

License

This repository includes an MIT license for open reuse.

File: Project_Documentation.md

Project Documentation

Executive Summary

This project is a full-stack retrieval-augmented generation (RAG) platform that combines a React/Next.js frontend with a FastAPI backend, MongoDB data storage, and a Mistral LLM gateway. The application enables authenticated users to upload documents, manage knowledge sources, perform safe conversational search, and review analytics.

Problem Statement

Knowledge workers need fast access to domain-specific answers from internal documents while avoiding AI hallucinations and prompt injection risks. This project provides a secure and user-friendly RAG system to surface relevant passages, return answer citations, and guard against malicious prompts.

Objectives

- Build a secure search and chat experience for enterprise documents.
- Use a modern web frontend with responsive interfaces and role-based admin pages.
- Implement a backend service with modular AI provider support and logging.
- Ensure transparent answer citations and operational monitoring.
- Prepare deployable artifacts for local and Render/Vercel cloud environments.

Scope

The implementation covers:

- Document ingestion and chunk-based retrieval
- Semantic search using a lightweight in-memory vector store
- Chat-based question answering with context-aware prompt construction
- User authentication, roles, and admin dashboards
- Audit logging, analytics, and feedback capture
- Deployment support via Docker Compose and Render/Vercel compatibility

Technology Stack

- Frontend: Next.js 15, React 19, TypeScript, Tailwind CSS
- Backend: FastAPI, Python 3.13, Pydantic, Motor (MongoDB async)
- Database: MongoDB for user, document, analytics, and audit data
- LLM Gateway: Mistral API as single provider
- DevOps: Docker Compose, Render deployment manifest, Vercel-ready frontend

Key Features

- Authenticated user registration, login, and password reset flows
- Document upload, search filters, and knowledge graph support
- Conversational question answering with citations and token usage
- Prompt injection detection on user queries
- Admin panels for documents, feedback, audit logs, analytics, and security
- Deployment-ready configuration for local dev and cloud

Implementation Summary

Backend

- ``backend/app/core/config.py`` manages environment settings and ensures Mistral-only provider flow.
- ``backend/app/services/llm_gateway.py`` builds prompt text and invokes Mistral via a single provider call.
- ``backend/app/services/rag.py`` controls retrieval, citation assembly, prompt injection detection, and query tracking.
- ``backend/app/db/vector.py`` implements a simple in-memory vector search layer for document chunks.

Frontend

- ``frontend/app/page.tsx`` and nested route pages provide UI for chat, documents, analytics, and admin tasks.
- ``frontend/components`` include reusable shell and page layout components.
- ``frontend/lib/api.ts`` centralizes API fetch functions and error handling.

Testing and Validation

- Backend unit tests verify security checks and core service behavior.
- Frontend build validation was confirmed with ``npm run build`` after the Next.js upgrade.
- Python syntax validation and dependency checks were verified for the updated backend runtime.

Results and Outcomes

- The app now supports a single trusted LLM provider (``Mistral``) with fallback removed.
- The Render backend runtime is pinned to Python 3.13 to avoid ``tokenizers`/Py03` build failures.
- Documentation is expanded to support deployment, architecture review, and submission packaging.

Challenges

- Ensuring compatibility between Render's Python runtime and LLM-related native dependencies.
- Removing multi-provider fallback logic while preserving a stable production-ready gateway.
- Documenting a broad full-stack system in a clear package suitable for academic evaluation.

Future Enhancements

- Replace the in-memory vector store with a production vector database such as Chroma or Pinecone.
- Add streaming responses for chat sessions and multi-turn conversation state.
- Enhance analytics with user behavior dashboards and heatmaps.
- Add end-to-end tests for UI flows and deployment smoke tests.

Conclusion

This repository is packaged as a deployable, maintainable RAG platform with a clear focus on security, citation transparency, and modern web stack integration.

File: System_Architecture.md

System Architecture

High-Level Architecture

The solution is structured into three main layers:

- Frontend
- Backend API
- Storage and Model Gateway

Frontend

- Built with Next.js 15 and React 19
- Provides authentication, document management, chat, analytics, and admin pages
- Communicates with the backend through REST API endpoints

Backend API

- FastAPI service exposing versioned routes under `backend/app/api/v1`
- Authenticates requests and enforces RBAC
- Coordinates retrieval, LLM calls, and audit logging

Storage

- MongoDB stores user records, documents, queries, feedback, analytics, and security logs
- A simple in-memory vector store is used for semantic search over ingested chunks

Component Diagram

```
```mermaid
graph TD
 A[User Browser] -->|HTTP / API| B[Next.js Frontend]
 B -->|REST API| C[FastAPI Backend]
 C -->|DB Access| D[(MongoDB)]
 C -->|LLM call| E[Mistral API]
 C -->|Search| F[In-memory Vector Store]
 D -->|Document Storage| F
 B -->|Static Assets| G[Public UI]
```

### ## Data Flow

1. A user logs in or registers via the frontend.
2. The frontend sends a request to the backend authentication endpoints.
3. Authorized users upload or manage documents.
4. Document content is chunked and stored in the vector search store.
5. A chat request triggers semantic search and context retrieval.
6. The backend builds a prompt, calls Mistral, and returns the answer with citations.
7. Queries, token usage, and security events are recorded in MongoDB.

### ## Backend Data Components

- `users`: authentication, roles, and profile data
- `documents`: uploaded documents metadata and storage references
- `queries`: chat queries with answers, citations, and token metrics
- `feedback`: user feedback on responses
- `audit\_logs`: system events and admin actions

- `security\_logs`: prompt injection and security incident tracking

## ## Vector Store

The vector store in `backend/app/db/vector.py` currently uses an in-memory chunk list.

- Chunks are scored by word overlap with the query.
- Filters are applied by metadata keys.
- The top N chunks are included in the LLM context.

## ## Sequence Diagram

```
```mermaid
```

```
sequenceDiagram
```

```
    participant U as User
```

```
    participant F as Frontend
```

```
    participant B as Backend
```

```
    participant V as Vector Store
```

```
    participant L as Mistral API
```

```
    participant M as MongoDB
```

```
    U->>F: Submit chat question
```

```
    F->>B: POST /chat
```

```
    B->>B: prompt injection check
```

```
    B->>V: search(query, filters)
```

```
    V-->>B: top chunks
```

```
    B->>L: prompt + context
```

```
    L-->>B: answer
```

```
    B->>M: record query and analytics
```

```
    B-->>F: answer + citations
```

```
    F-->>U: display response
```

```
```
```

## ## Deployment Architecture

The project supports two deployment modes:

- Local development with `docker-compose.yml`
- Cloud deployment using `infra/render.yaml` for the backend and Vercel-ready frontend

### ### Render backend details

- `backend/runtime.txt` pins Python to version 3.13
- The service uses the FastAPI backend and MongoDB connection strings
- Environment variables are managed through Render secrets

## ## Security and Reliability

- Backend verifies prompt safety before any vector search or model call.
- Audit and security logs capture suspicious user actions.
- Token usage and latency metrics support cost monitoring.

## ## Future Architecture Improvements

- Replace the in-memory vector store with a dedicated vector database
- Add caching for frequently accessed document content
- Introduce streaming responses and conversation state persistence

## File: User\_Manual.md

# User Manual

### ## Purpose

This manual explains how to install, configure, run, and use the application from both developer and end-user perspectives.

### ## Prerequisites

- Node.js 20+ and npm
- Python 3.13
- Docker and Docker Compose (for local containerized deployment)
- MongoDB instance or connection string
- Mistral API key

### ## Local Setup

1. Clone the repository.
2. Copy ``.env.example`` to ``.env`` and populate required values.
3. Install backend dependencies:
  - ``cd backend``
  - ``python -m pip install -r requirements.txt``
4. Install frontend dependencies:
  - ``cd ../frontend``
  - ``npm install``

### ## Configuration

Required environment variables are documented in ``.env.example``.

- ``MISTRAL_API_KEY``: API key for the Mistral model.
- ``MISTRAL_MODEL``: Example value: ``mistral-large-latest``.
- ``MONGO_URI``: Connection string for MongoDB.
- ``NEXT_PUBLIC_API_BASE_URL``: Base URL for frontend API calls.
- ``SECRET_KEY``: Secret key for backend sessions and security.

### ## Running Locally

#### ### Backend

From ``backend``:

```
```bash
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```
```

#### ### Frontend

From ``frontend``:

```
```bash
npm run dev
```
```

#### ### Docker Compose

From the repo root:

```
```bash
```

```
docker compose up --build
...
```

This starts MongoDB, the backend, and the frontend together.

Application Workflow

Authentication

- Register a new user or sign in with existing credentials.
- Access protected pages if authenticated.

Document Management

- Upload documents in the documents area.
- Documents are chunked and stored for semantic search.
- Admin users can review and manage documents.

Conversational Search

- Use the chat page to ask natural language questions.
- The backend retrieves the most relevant document chunks.
- Mistral generates an answer grounded in those chunks.
- Citations are displayed to show source evidence.

Feedback and Evaluation

- Users can submit feedback on generated responses.
- Feedback and evaluation pages enable quality review.

Analytics and Admin

- Analytics panels show usage metrics, costs, and model performance.
- Audit logs record user actions and system events.
- Security logs capture prompt injection warnings.

Troubleshooting

Common Issues

- `Mistral` calls fail: verify `MISTRAL\_API\_KEY` and `MISTRAL\_MODEL`.
- `MongoDB` connection errors: verify `MONGO\_URI`.
- Frontend build errors: update packages and ensure `NEXT\_PUBLIC\_API\_BASE\_URL` is set.
- Backend import or runtime errors: ensure Python 3.13 and dependencies are installed.

Deployment Tips

- Pin the backend runtime with `backend/runtime.txt` to avoid Python 3.14 compatibility issues.
- Use environment secrets instead of committing `.env`.

Admin Notes

- Admin access is handled in the frontend admin route structure.
- Audit and security logs should be reviewed regularly for suspicious activity.
- Use the developer tools in the browser to inspect API responses when troubleshooting.

File: Deployment_Guide.md

Deployment Guide

Overview

This guide describes how to deploy the project locally and in cloud environments. The repository includes support for Docker Compose and Render.

Local Deployment

Using Docker Compose

From the repository root:

```
```bash
docker compose up --build
```
```

This will start:

- `mongo`: MongoDB database
- `backend`: FastAPI service
- `frontend`: Next.js application

Verifying Local Deployment

- Frontend: `http://localhost:3000`
- Backend: `http://localhost:8000`

Cloud Deployment

Render Backend

The backend deployment is configured through `infra/render.yaml`.

Key deployment considerations:

- `backend/runtime.txt` pins Python to version 3.13.
- Use Render secrets for all sensitive values.
- Set `NEXT\_PUBLIC\_API\_BASE\_URL` to your backend service URL.

Backend Environment Variables

The backend requires:

- `MISTRAL\_API\_KEY`
- `MISTRAL\_MODEL`
- `MONGO\_URI`
- `SECRET\_KEY`
- `NEXT\_PUBLIC\_API\_BASE\_URL`

Frontend Deployment

The frontend is Vercel-ready and can be deployed from the `frontend` directory.

- Ensure `NEXT\_PUBLIC\_API\_BASE\_URL` is configured as a Vercel environment variable.
- Confirm the output tracing configuration in `frontend/next.config.ts`.

Runtime Compatibility

- The backend uses Python 3.13 to avoid native dependency build issues with `tokenizers`

and `Py03`.

- `backend/runtime.txt` is included to enforce this version on Render.

Production Hardening

- Use HTTPS for all API traffic.
- Store secrets in environment management rather than `.env` files.
- Enable authentication and role-based access controls in the deployment configuration.
- Review audit and security logs after deployment.

Deploying with Render

1. Create a Render Web Service for the backend.
2. Connect the repository and set the build command.
3. Use `backend/runtime.txt` to lock Python.
4. Add the required environment variables in Render.

Deploying the Frontend on Vercel

1. Link the frontend directory to Vercel.
2. Set `NEXT\_PUBLIC\_API\_BASE\_URL` to the deployed backend URL.
3. Ensure the project root contains `frontend/next.config.ts` with `outputFileTracingRoot`.

Verification

After deployment:

- Visit the frontend URL and log in.
- Perform a sample chat query and confirm the backend returns citations.
- Check the backend service logs for startup errors.

Notes

- If the backend build fails on Render, verify Python version and dependency compatibility.
- If the frontend cannot reach the backend, confirm the API base URL environment variable and CORS configuration.

File: Prompt_Document.md

Prompt Engineering Document

Overview

This application uses a single LLM provider, Mistral, to answer user queries from retrieved document context.

Prompt design focuses on:

- grounding answers in retrieved citations
- limiting hallucinations by exposing only relevant context
- rejecting malicious prompt injection attempts
- preserving a clean question/answer loop for analytics

Final Prompt Template

The production prompt is constructed in `backend/app/services/llm\_gateway.py`.

Template

```

You are a knowledgeable assistant. Use the following context to answer the user question. Provide clear, concise, and accurate responses.

Context:

{context}

Question:

{question}

```

Prompt Engineering Strategy

- `context`: retrieved document chunks from prior semantic search.
- `question`: the end user query.
- No additional system or developer instructions are injected beyond the user-oriented prompt content.
- The model is explicitly asked to answer using only the provided context.

Safety and Injection Protection

The backend detects prompt injection in every incoming question before retrieval or LLM invocation.

Injection patterns

- `ignore previous`
- `system prompt`
- `developer message`
- `reveal secrets`

If a query matches any pattern, the request is rejected with a 400 error and a security log entry is created.

Notes on Prompt Variants

- Only one prompt variant is used in the current codebase.
- The prompt is intentionally simple to maximize transparency and reduce unexpected model behavior.

- Token usage is tracked for both prompt and completion tokens to estimate cost.

Prompt Evaluation

- The prompt is designed for a retrieval-augmented scenario, with the model conditioned on only the relevant chunks.
- The generated answer is returned with citation metadata from the retrieved chunks.

Future Prompt Enhancements

- Add an explicit instruction to cite source filename and chunk index in the response.
- Use a schema-based output format to make downstream UI parsing easier.
- Add query rewriting and context truncation when documents grow larger.

File: Presentation_Content.md

Presentation Content

Project Title

Enterprise RAG Platform with Secure LLM Augmented Search

Slide 1: Introduction

- Problem: Knowledge discovery is hard in document-heavy environments.
- Solution: A full-stack RAG platform with authenticated access, document management, and LLM-powered conversational search.
- Goal: Demonstrate a deployable, secure, citation-aware QA system.

Slide 2: Architecture

- Frontend: Next.js + TypeScript + Tailwind
- Backend: FastAPI + Python
- Storage: MongoDB
- LLM Gateway: Mistral
- Deployment: Docker Compose + Render/Vercel

Slide 3: Key Features

- Secure login and user roles
- Document upload and management
- Conversational retrieval and citation
- Prompt injection detection and audit logging
- Analytics and admin dashboards

Slide 4: Technical Highlights

- Single-provider LLM gateway simplified to Mistral
- Python 3.13 runtime pin to avoid build issues
- In-memory vector search for fast semantic retrieval
- Transparent metrics for token usage and cost estimation
- Clean separation of frontend and backend concerns

Slide 5: Demo Flow

1. User logs in or registers.
2. User uploads documents or selects a dataset.
3. User asks a question in the chat interface.
4. Backend retrieves relevant chunks, calls Mistral, and returns a grounded answer.
5. User sees citations and optionally submits feedback.

Slide 6: Challenges and Resolutions

- Issue: Render backend build failed due to Python 3.14 / `tokenizers` mismatch.
- Fix: Pinned backend runtime to Python 3.13 in `backend/runtime.txt`.
- Issue: Multi-provider fallback complexity.
- Fix: Simplified provider stack to Mistral only.

Slide 7: Results

- Stable frontend build with Next.js 15.5.19
- Working backend architecture with a single model provider
- Comprehensive deployment and documentation package

Slide 8: Future Work

- Integrate a true vector database
- Add conversational state and multi-turn memory
- Improve analytics and evaluation dashboards
- Support additional secure LLM providers if needed

Slide 9: Conclusion

- This project delivers a ready-to-deploy RAG system for enterprise knowledge access.
- It emphasizes security, transparency, and maintainability.
- The codebase is documented and prepared for academic submission.

File: Final_Submission_Checklist.md

Final Submission Checklist

Repository Contents

- [] `README.md` updated with project overview and docs links
- [] `docs/` folder includes architecture, deployment, user manual, prompt documentation, and presentation content
- [] `infra/render.yaml` included for backend deployment
- [] `docker-compose.yaml` included for local development
- [] `.env.example` updated for current Mistral-only configuration
- [] `backend/runtime.txt` present for Python 3.13 runtime pin
- [] `LICENSE` included if required by submission rules

Documentation

- [] `docs/Project\_Documentation.md` created
- [] `docs/System\_Architecture.md` created
- [] `docs/User\_Manual.md` created
- [] `docs/Deployment\_Guide.md` created
- [] `docs/Prompt\_Document.md` created
- [] `docs/Presentation\_Content.md` created
- [] `docs/Final\_Submission\_Checklist.md` created
- [] `screenshots/SCREENSHOT\_CHECKLIST.md` created

Deployment and Verification

- [] Local development run verified with Docker Compose
- [] Frontend build verified with `npm run build`
- [] Backend Python compatibility verified with Python 3.13
- [] Render backend runtime pinned via `backend/runtime.txt`
- [] API endpoints documented and ready for use

Review

- [] Confirm no remaining fallback model configuration in backend
- [] Confirm prompt injection detection is enabled
- [] Confirm docs reference current architecture and deployment plan
- [] Confirm README links to new documentation files
- [] Confirm `.env.example` only includes required values

File: SCREENSHOT_CHECKLIST.md

Screenshot Checklist

Required Screenshots

- [] Login page
- [] Registration page
- [] Dashboard or home page after login
- [] Document upload page
- [] Document listing or management page
- [] Chat / conversational search page
- [] Answer with citations displayed
- [] Feedback submission or evaluation page
- [] Analytics dashboard page
- [] Audit logs or security logs page
- [] Deployment status or environment configuration page

Notes

- Capture the UI state with real data where possible.
- Include visible timestamps or labels to show feature behavior.
- If using staging or deployed environment, include the public URL.